Insertion Sort

```
#include <iostream>
using namespace std;

void insertionSort(int arr[], int n) {
   for (int i = 1; i<n; i++) {
      int temp = arr[i];
      int j = i - 1;

      while (j >= 0 && arr[j] > temp) {
         arr[j + 1] = arr[j];
         j--;
      }
      arr[j + 1] = temp;
   }
}

void printArray(int arr[], int n) {
   for (int i = 0; i < n; i++) {
      cout<<arr[i] << " ";
   }
   cout<<endl;
}

int main() {
   int n;
   cout << "Enter the number of elements: ";
   cin >> n;

   int arr[n];
   cout << "Enter the elements: ";
   for (int i = 0; i < n; i++) {
      cin>>arr[i];
   }

   insertionSort(arr, n);
   cout <<"Sorted array: ";
   printArray(arr, n);

   return 0;
}
```

Selection Sort

```
#include <iostream>
using namespace std;

void selectionSort(int arr[], int n) {
   for (int i = 0; i < n - 1; i++) {
      int min = i;
      for (int j = i + 1; j < n; j++) {
         if (arr[j] < arr[min]) {
            min = j;
         }
      }
      if (min != i) {
         swap(arr[i], arr[min]);
      }
   }
}

int main() {
   int n;
   cout << "Enter number of elements: ";
   cin >> n;

   int arr[n];
   cout << "Enter the elements: ";
   for (int i = 0; i < n; i++) {
      cin >> arr[i];
   }
   selectionSort(arr, n);
   cout << "Sorted elements are: ";
   for (int i = 0; i < n; i++) {
      cout << arr[i] << " ";
   }

   return 0;
}
```

Bubble Sort

```
#include <iostream>
using namespace std;

void bubbleSort(int arr[], int n) {
   for (int i = 0; i < n - 1; i++) {
      int flag = 0;
      for (int j = 0; j < n - i - 1; j++) {
         if (arr[j] > arr[j + 1]) {
            swap(arr[j], arr[j + 1]);
            flag = 1;
         }
      }
      if (flag==0)
         break;
   }
}

int main() {
   int n;
   cout << "Enter number of elements: ";
   cin >> n;

   int arr[n];
   cout << "Enter the elements: ";
   for (int i = 0; i < n; i++) {
      cin >> arr[i];
   }
   bubbleSort(arr, n);
   cout << "Sorted elements are: ";
   for (int i = 0; i < n; i++) {
      cout << arr[i] << " ";
   }

   return 0;
}
```

Merge Sort

```
#include <iostream>
using namespace std;

void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int* leftArr = new int[n1];
    int* rightArr = new int[n2];

    for (int i = 0; i < n1; i++) leftArr[i] = arr[left + i];
    for (int j = 0; j < n2; j++) rightArr[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        arr[k++] = (leftArr[i] <= rightArr[j]) ? leftArr[i++] : rightArr[j++];
    }
    while (i < n1) arr[k++] = leftArr[i++];
    while (j < n2) arr[k++] = rightArr[j++];

    delete[] leftArr;
    delete[] rightArr;
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() {
    int arr[] = {6, 4, 2, 1, 9, 8, 3, 5, 12, 10, 11};
    int size = sizeof(arr) / sizeof(arr[0]);

    mergeSort(arr, 0, size - 1);

    cout << "Sorted array: ";
    for (int i = 0; i < size; i++) cout << arr[i] << " ";
    cout << endl;

    return 0;
}
```

Quick Sort

```
#include <iostream>
using namespace std;

void quickSort(int arr[], int low, int high);
int partition(int arr[], int low, int high);

int main() {
   int arr[] = {7, 6, 10, 5, 9, 2, 1, 15, 7};
   int n = sizeof(arr)/sizeof(arr[0]);
   quickSort(arr, 0, n - 1);
   cout<<"Sorted array: ";
   for (int i = 0; i < n; i++) {
      cout<<arr[i]<< " ";
   }
   cout<<endl;
   return 0;
}

void quickSort(int arr[], int low, int high) {
   if (low < high) {
      int pivotIndex = partition(arr, low, high);
      quickSort(arr, low, pivotIndex - 1);
      quickSort(arr, pivotIndex + 1, high);
   }
}

int partition(int arr[], int low, int high) {
   int pivot = arr[low];
   int left = low + 1;
   int right = high;
   while(left <= right) {
      while(left<=right&&arr[left]<=pivot) {
         left++;
      }
      while(left <= right && arr[right]>pivot) {
         right--;
      }
      if (left < right) {
         swap(arr[left],arr[right]);
      }
   }
   swap(arr[low], arr[right]);
   return right;
}
```

Counting Sort

```
#include <iostream>
using namespace std;

void countingSort(int arr[], int size) {
   int maxVal = arr[0];
   for (int i = 1; i < size; i++) {
      if (arr[i] > maxVal) {
         maxVal = arr[i];
      }
   }
   int count[maxVal + 1] = {0};
   for (int i = 0; i < size; i++) {
      count[arr[i]]++;
   }
   int index = 0;
   for (int i = 0; i <= maxVal; i++) {
      while (count[i] > 0) {
         arr[index++]=i;
         count[i]--;
      }
   }
}
int main() {
   int arr[] = {6, 4, 2, 1, 9, 8, 3, 5, 12, 10, 11};
   int size = sizeof(arr)/sizeof(arr[0]);
   countingSort(arr, size);
   for (int i=0; i<size; i++) {
      cout<<arr[i]<< " ";
   }
   cout<<endl;

   return 0;
}
```

Linear Search

```
#include <iostream>
using namespace std;

int linearSearch(int a[], int size, int target) {
   for (int i=0; i<size; i++) {
      if (a[i]==target) {
         return i;
      }
   }
   return -1;
}

int main() {
   int a[]={10, 15, 30, 70, 80, 60, 20, 90, 40};
   int size = sizeof(a)/sizeof(a[0]);
   int target;

   cout<<"Enter the number to search: ";
   cin>>target;

   int result=linearSearch(a,size,target);

   if (result!=-1) {
      cout<<"Data found at index: "<<result<<endl;
   } else {
      cout<<"Data not found in the array." <<endl;
   }

   return 0;
}
```

Binary Search

```
#include <iostream>
using namespace std;

int binarySearch(int arr[], int size, int key) {
    int left = 0;
    int right = size - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == key) {
            return mid;
        } else if (arr[mid] < key) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1;
}

int main() {
    int arr[] = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35, 37,
39, 41, 43, 45, 47, 49};
    int size = sizeof(arr) / sizeof(arr[0]);

    int keys[3];
    cout << "Enter 3 keys to search: ";
    for (int i = 0; i < 3; i++) {
        cin >> keys[i];
    }

    for (int key : keys) {
        int index = binarySearch(arr, size, key);
        if (index != -1) {
            cout << "Key " << key << " found at index: " << index << endl;
        } else {
            cout << "Key " << key << " not found." << endl;
        }
    }

    return 0;
}
```

DFS

```
#include <iostream>
using namespace std;

void DFS(int graph[10][10], int nodes, int start) {
   int stack[10];
   int top = -1;
   int visited[10] = {0};
   stack[++top] = start;

   cout << "DFS Traversal: ";
   while (top != -1) {
      int current = stack[top--];
      if (visited[current] == 0) {
         cout << current << " ";
         visited[current] = 1;

         for (int i = 0; i < nodes; i++) {
            if (graph[current][i] == 1 && visited[i] == 0) {
               stack[++top] = i;
            }
         }
      }
   }
   cout << endl;
}

int main() {
   int nodes;
   cout << "Enter the number of nodes: ";
   cin >> nodes;

   int graph[10][10] = {0};

   cout << "Enter the adjacency matrix (0 or 1):" << endl;
   for (int i = 0; i < nodes; i++) {
      for (int j = 0; j < nodes; j++) {
         cin >> graph[i][j];
      }
   }

   int start;
   cout << "Enter the starting node for DFS: ";
   cin >> start;

   DFS(graph, nodes, start);
   return 0;
}
```

BFS

```
#include <iostream>
using namespace std;

void BFS(int graph[10][10], int nodes, int start) {
    bool visited[10] = {false};
    int queue[10];
    int front = 0, rear = 0;

    visited[start] = true;
    queue[rear++] = start;
    cout << "BFS Traversal: ";

    while (front != rear) {
        int current = queue[front++];
        cout << current << " ";

        for (int i = 0; i < nodes; i++) {
            if (graph[current][i] == 1 && !visited[i]) {
                visited[i] = true;
                queue[rear++] = i;
            }
        }
    }
    cout << endl;
}

int main() {
    int nodes;
    cout << "Enter the number of nodes: ";
    cin >> nodes;

    int graph[10][10] = {0};

    cout << "Enter the adjacency matrix (0 or 1):" << endl;
    for (int i = 0; i < nodes; i++) {
        for (int j = 0; j < nodes; j++) {
            cin >> graph[i][j];
        }
    }

    int start;
    cout << "Enter the starting node for BFS: ";
    cin >> start;

    BFS(graph, nodes, start);
    return 0;
}
```